

# The Source Code Book

Every important module, its real current code, and a block-by-block walkthrough of why it's written that way — not just what it's used for. Read top to bottom in this order and you're following the exact path one repository takes through the system, from a raw `git clone` to a searchable answer.

---

---

## Part 1 — Foundations

One file that every other module imports from. Start here.

src/codeseek/config.py

## config.py

The single source of truth for paths, the corpus list, and every "which files count as code/docs/ignored" rule. Nothing in this file does work — it only declares constants other modules import.

```
ROOT_DIR = Path(__file__).resolve().parents[2]
DATA_DIR = ROOT_DIR / "data"
REPOS_DIR = DATA_DIR / "repos"
QDRANT_PATH = DATA_DIR / "qdrant_storage"
CORPUS_NAME = "codeseek"

@dataclass(frozen=True)
class RepoSpec:
    name: str
    url: str
    language: str

CORPUS: list[RepoSpec] = [
    RepoSpec("httpx", "https://github.com/encode/httpx.git", "python"),
    # ... 23 more repos, Python and JS/TS
]

CODE_EXTENSIONS = {".py", ".js", ".jsx", ".ts", ".tsx", ".mjs", ".cjs"}
DOC_EXTENSIONS = {".md", ".rst"}
IGNORED_DIR_NAMES = {".git", "node_modules", "__pycache__", ...}
NON_ENGLISH_DOC_LOCALES = {"de", "es", "fr", "hi", "ja", ...}
```

## Walkthrough

1. **ROOT\_DIR** via **Path(\_\_file\_\_).resolve().parents[2]**. This file lives at **src/codeseek/config.py** — two **.parent** hops up lands on the project root, regardless of what directory you happened to run Python from. Every other path (**DATA\_DIR**, **REPOS\_DIR**, **QDRANT\_PATH**) is built relative to that, so the whole project works the same whether you run it from the repo root, a subfolder, or a different machine entirely.
2. **RepoSpec** is a **frozen dataclass** — just three fields (name, git URL, nominal language). "Frozen" means immutable: once created, nothing can accidentally mutate a **RepoSpec** mid-pipeline. It's the one object that represents "a repo we might index," passed by reference through **clone\_or\_update** → **index\_repo** → the chunkers.

3. **CORPUS\_NAME** is deliberately one constant, not one per repo. Every repo shares two Qdrant collections (**codeseek\_\_general**, **codeseek\_\_code**) — repos are distinguished by a **repo** field on each stored point, not by separate collections. That's the single design decision that makes cross-repo search ("find the best answer across everything") and single-repo search ("only look in httpx") the exact same code path with just a filter difference.
4. **NON\_ENGLISH\_DOC\_LOCALES** exists because of a real bug found mid-project: fastapi's docs are translated into ~20 languages, and without this set, every translation gets walked, chunked, and embedded as if it were unique content. See **walker.py** below for where it's used.

## Part 2 — Ingestion

Turning a GitHub URL into a list of files worth chunking.

## clone.py

Gets a repo's source onto local disk. That's the entire job — two functions, no chunking, no embedding logic here.

```
def infer_repo_name(url: str) → str:
    return url.rstrip("/").rsplit("/", 1)[-1].removesuffix(".git")

def clone_or_update(repo: RepoSpec, dest_dir: Path = REPOS_DIR) → Path:
    repo_path = dest_dir / repo.name

    if repo_path.exists():
        subprocess.run(
            ["git", "-C", str(repo_path), "pull", "--depth", "1"],
            check=True, capture_output=True, text=True,
        )
        return repo_path

    dest_dir.mkdir(parents=True, exist_ok=True)
    subprocess.run(
        ["git", "clone", "--depth", "1", repo.url, str(repo_path)],
        check=True, capture_output=True, text=True,
    )
    return repo_path
```

### Walkthrough

1. **infer\_repo\_name** turns `https://github.com/psf/black.git` into `black` — strip a trailing slash, take the last `/`-separated segment, drop a trailing `.git`. This is what lets the "index any repo" UI form work with just a URL and no required name field.
2. **clone\_or\_update** checks `repo_path.exists()` first. If the repo was already cloned, it does a shallow `git pull` instead of re-cloning from scratch — cheap idempotency, so running the ingestion script twice doesn't waste bandwidth.
3. **--depth 1 on both the clone and the pull.** A shallow clone — only the latest commit, no history — because nothing downstream (chunking, embedding) needs git history, only the current file contents. This is a real, deliberate speed/disk trade-off, not an oversight.
4. **check=True, capture\_output=True.** `check=True` makes `subprocess.run` raise `CalledProcessError` on a non-zero exit code (a bad URL, no network) instead of

silently continuing with a half-cloned or missing repo — a real instance of this project's "fail loud, not silent" principle. `capture_output=True` keeps git's own stdout/stderr out of our logs unless something actually goes wrong.

#### CONNECTS TO

Called from `scripts/ingest.py` (bulk, all of `CORPUS`) and from `api/app.py`'s `/ingest` route (one ad-hoc repo at a time).

## walker.py

Walks every file in a cloned repo and decides, per file, one of three outcomes: it's code, it's docs, or it's ignored. Nothing here reads file *contents* for routing — only paths and extensions.

```
def _route(path: Path) → Category | None:
    if path.suffix in CODE_EXTENSIONS:
        return "code"
    if path.suffix in DOC_EXTENSIONS or path.name.upper().startswith("README"):
        return "docs"
    return None

def _is_translated_doc(path: Path) → bool:
    parts = path.parts
    for i, part in enumerate(parts[:-1]):
        if part == "docs" and parts[i + 1] in NON_ENGLISH_DOC_LOCALES:
            return True
    return False

def walk_repo(repo_path: Path, repo_name: str, language: str) → Iterator[FileRecord]:
    for path in repo_path.rglob("*"):
        if not path.is_file():
            continue
        if any(part in IGNORED_DIR_NAMES for part in path.parts):
            continue
        if _is_translated_doc(path.relative_to(repo_path)):
            continue

        category = _route(path)
        if category is None:
            continue

        try:
            size = path.stat().st_size
        except OSError:
            continue
        if size == 0:
            continue

        yield FileRecord(
            repo=repo_name, language=language, category=category,
            path=path.relative_to(repo_path).as_posix(), size_bytes=size,
        )
```

1. `walk_repo` is a generator (uses `yield`, returns an `Iterator`) — it doesn't build a big list in memory up front. For a repo with thousands of files, this means the caller can start processing the first file before the last one has even been checked.
2. **Filter order matters for speed.** `is_file()` first (skip directories instantly), then `IGNORED_DIR_NAMES` (skip all of `node_modules` without even checking the extension), then the translated-docs check, then routing. Cheap checks run before expensive ones.
3. `path.relative_to(repo_path).as_posix()`. This one line fixed a real bug: on Windows, a plain `str(path)` uses backslashes (`httpx\_auth.py`), but the rest of the system (ground-truth JSON, ID generation) assumes forward slashes. `.as_posix()` forces forward slashes regardless of OS, so a chunk's identity is the same whether it was indexed on Windows or Linux.
4. **Zero-byte files are skipped** (`if size == 0: continue`) — an empty file has nothing to chunk and would just be dead weight in the manifest.
5. `_is_translated_doc` **only matches a locale directly under a literal "docs" folder** — it doesn't blanket-ban any folder named `de` or `ja` anywhere in a repo (which could be a legitimate code namespace), only `docs/de/ ...` specifically.

## Part 3 — Chunking

The core differentiator: turning a file into semantically whole pieces instead of fixed-size text windows. Four files, one shared algorithm underneath two different chunkers.

---

src/codeseek/chunking/shared.py

## shared.py — the engine both chunkers run on

Neither the Python chunker nor the JS/TS chunker implements its own size-limiting logic — both call into `pack_units`, defined once here. This is the file to understand first, because everything else in chunking is "get a list of atomic units, then hand them to this."



```

def count_tokens(text: str) → int:
    return len(_ENCODING.encode(text)) # tiktoken's cl100k_base encoding

def _hard_split_by_words(text: str, max_tokens: int) → list[str]:
    """Last resort: a single unit alone exceeds max_tokens. Cut on
    whitespace only -- never mid-word/mid-token."""
    words = text.split(" ")
    pieces: list[str] = []
    current: list[str] = []
    for word in words:
        candidate = current + [word]
        if current and count_tokens(" ".join(candidate)) > max_tokens:
            pieces.append(" ".join(current))
            current = [word]
        else:
            current = candidate
    if current:
        pieces.append(" ".join(current))
    return pieces

def pack_units(units, max_tokens, overlap_ratio, joiner) → list[tuple[int, int, str]]:
    results: list[tuple[int, int, str]] = []
    current: list[int] = []

    def flush() → None:
        nonlocal current
        if current:
            text = joiner.join(units[i] for i in current)
            results.append((current[0], current[-1], text))
            overlap_count = max(1, round(len(current) * overlap_ratio)) if current and overlap_ratio else 1
            current = current[-overlap_count:] if overlap_count else []

    for idx, unit in enumerate(units):
        if count_tokens(unit) > max_tokens:
            flush()
            results.extend((idx, idx, piece) for piece in _hard_split_by_words(unit, max_tokens))
            continue

        prospective = current + [idx]
        if current and count_tokens(joiner.join(units[i] for i in prospective)) > max_tokens:
            flush()
            current.append(idx)

    flush()
    return results

```

## Walkthrough — this is the part worth really understanding

1. **A "unit" is the atom the caller defines** — for the Python/JS chunkers it's a single *line* of source code (so a split never cuts a statement in half); for the docs chunker it's

a single *sentence*. `pack_units` itself doesn't know or care which — it just greedily packs whatever list of strings you give it.

2. **The greedy loop.** For each unit, in order: if adding it to the current chunk would push the token count over `max_tokens`, `flush()` the current chunk first, then start a new one with this unit. This is a classic greedy bin-packing approach — simple, fast, and because units are indivisible, a flush point always falls on a real line/sentence boundary.
3. **`flush()` is a closure with `nonlocal current`.** It does two things: (a) if there's anything accumulated, emit it as a result tuple `(start_index, end_index, text)` — the indices let a caller map back to real line numbers; (b) reset `current` to the *overlap tail* of what was just flushed, not an empty list. That's how overlap works: the next chunk starts by re-including the last `overlap_ratio` fraction of the previous one.
4. **The one-unit-too-big escape hatch.** If a single unit (one enormous line, or one enormous sentence) alone exceeds `max_tokens`, there's no way to include it whole. Rather than crash or silently drop it, `_hard_split_by_words` is called as a last resort — it cuts on whitespace, never mid-word. This is the "graceful degradation" this project keeps choosing on purpose: the worst case is an imperfect split, never a failure.
5. **Why indices, not just text.** Returning `(start_index, end_index, text)` instead of just `text` is what lets `code_chunker.py` compute real `start_line`/`end_line` numbers for an oversized function's split parts — the caller adds the node's own starting line to the relative index.

---

`src/codeseek/chunking/code_chunker.py`

## **code\_chunker.py — Python's own parser does the hard part**

One chunk per top-level function or class, using Python's built-in `ast` module — no third-party parser needed for Python specifically, because the standard library already ships one.

```

_SYMBOL_TYPE = {
    ast.FunctionDef: "function",
    ast.AsyncFunctionDef: "async_function",
    ast.ClassDef: "class",
}

def chunk_python_source(source: str, repo: str, path: str) → list[CodeChunk]:
    try:
        tree = ast.parse(source)
    except SyntaxError:
        return []

    lines = source.splitlines()
    chunks: list[CodeChunk] = []

    for node in ast.iter_child_nodes(tree):
        symbol_type = _SYMBOL_TYPE.get(type(node))
        if symbol_type is None:
            continue

        start_line = node.lineno
        if node.decorator_list:
            start_line = node.decorator_list[0].lineno
        end_line = node.end_lineno

        node_lines = lines[start_line - 1 : end_line]
        text = "\n".join(node_lines)

        if count_tokens(text) ≤ MAX_CHUNK_TOKENS:
            chunks.append(CodeChunk(
                repo=repo, language="python", path=path,
                symbol_name=node.name, symbol_type=symbol_type,
                start_line=start_line, end_line=end_line, text=text,
            ))
            continue

        packed = pack_units(node_lines, MAX_CHUNK_TOKENS, OVERSIZED_OVERLAP_RATIO, joiner="\n")
        for part_num, (rel_start, rel_end, piece_text) in enumerate(packed, start=1):
            chunks.append(CodeChunk(
                repo=repo, language="python", path=path,
                symbol_name=f"{node.name}##part{part_num}", symbol_type=symbol_type,
                start_line=start_line + rel_start, end_line=start_line + rel_end,
                text=piece_text,
            ))

    return chunks

```

## Walkthrough

1. **`ast.parse(source)`** turns the raw source text into a tree of real structural nodes. This is exactly what Python's own interpreter does before running your code — we're

reusing the interpreter's own understanding of "what is a function," not guessing from indentation or regex.

2. `try/except SyntaxError: return []`. If the file doesn't parse (a work-in-progress file, a Python 2 file, a template with non-Python syntax), the whole file is simply skipped — no chunks, no crash, no effect on any other file. This is the code-chunker's version of graceful degradation.
3. `ast.iter_child_nodes(tree)` — **only *direct* children of the module**. This is deliberate: it means a method defined inside a class is *not* visited separately — it stays embedded in its class's single chunk. A module-level function is chunked on its own. This matches "one chunk = one meaningful unit a person would ask about."
4. **Decorators shift the start line.** `node.lineno` points at the `def`/`class` keyword itself, not any `@decorator` above it. Checking `node.decorator_list` and using the first decorator's line number instead means `@app.get("/users")` stays attached to the function it decorates, instead of being silently dropped or ending up as an orphaned line in the previous chunk.
5. **The oversized path reuses `pack_units` with `joiner="\n"`** (not `" "`, which the docs chunker uses) — because these units are source lines, and joining lines with newlines instead of spaces is what keeps the split code syntactically sane to read.

#### IN YOUR OWN WORDS

"I don't write my own Python parser — I use the same `ast` module the Python interpreter itself is built on, walk only the top-level nodes, and slice the original source text by line number."

---

```
src/codeseek/chunking/js_ts_chunker.py
```

## **js\_ts\_chunker.py — the same idea, for languages Python has no parser for**

JavaScript and TypeScript don't have a parser built into Python, so this uses **tree-sitter** (the same parsing engine GitHub itself uses for code navigation) to get the equivalent of an AST, then applies the exact same "top-level nodes only, slice by position" strategy as the Python chunker.

```

_LANGUAGE_BY_SUFFIX = {
    ".js": ("javascript", Language(tsjavascript.language())),
    ".mjs": ("javascript", Language(tsjavascript.language())),
    ".ts": ("typescript", Language(tstypescript.language_typescript())),
    ".tsx": ("typescript", Language(tstypescript.language_tsx())),
}

def _declarator_symbol(node: Node) → tuple[str, str] | None:
    """A single `const foo = () => {}` declarator."""
    declarators = [c for c in node.named_children if c.type == "variable_declarator"]
    if len(declarators) != 1:
        return None
    declarator = declarators[0]
    value = declarator.child_by_field_name("value")
    name = declarator.child_by_field_name("name")
    if value is None or name is None or value.type not in _FUNCTION_VALUE_TYPES:
        return None
    return name.text.decode("utf-8"), "function"

def chunk_js_ts_source(source: str, repo: str, path: str, suffix: str) → list[CodeChunk]:
    langinfo = _LANGUAGE_BY_SUFFIX.get(suffix)
    if langinfo is None:
        return []
    language_name, ts_language = langinfo

    parser = Parser(ts_language)
    tree = parser.parse(source.encode("utf-8"))
    lines = source.splitlines()
    chunks: list[CodeChunk] = []

    for node in tree.root_node.named_children:
        slice_node = node
        target = node

        if node.type == "export_statement":
            decl = node.child_by_field_name("declaration")
            if decl is not None:
                target = decl
            else:
                value = node.child_by_field_name("value")
                if value is not None and value.type in _FUNCTION_VALUE_TYPES | {"class"}:
                    classified = ("default", "class" if value.type == "class" else "function")
                    chunks.extend(_emit(slice_node, classified, lines, repo, language_name, path))
                    continue

        classified = _classify(target)
        if classified is None:
            continue
        chunks.extend(_emit(slice_node, classified, lines, repo, language_name, path))

    return chunks

```

## Walkthrough

1. **Four grammars, mapped by extension.** `.js`/`.mjs`/`.cjs`/`.jsx` all use the plain JavaScript grammar (which already understands JSX); `.ts` uses the TypeScript grammar; `.tsx` uses a distinct TSX grammar. Each is its own compiled parser — picking the wrong one for a file would misparse TypeScript-only syntax like type annotations.
2. **Four patterns are recognized at the top level:** a plain `function foo() {}` (`function_declaration`), a `class Foo {}` (`class_declaration`), the extremely common `const foo = () => {}` arrow pattern (via `_declarator_symbol`), and any of those wrapped in `export`/`export default`.
3. **The `export_statement` branch is the trickiest part.** If it wraps a named declaration (`export function foo() {}`), we classify the *inner* node (`target = decl`) for the symbol name/type, but slice the text starting from the **outer** `export_statement` node (`slice_node = node`) — so the word "export" stays attached to the chunk's text instead of being silently dropped. If it's an anonymous default export (`export default () => {}`), there's no name to extract, so the symbol is just called `"default"`.
4. **`_declarator_symbol` deliberately only handles a *single* declarator.** `const a = 1, b = 2;` (two declarators in one statement) returns `None` and is skipped rather than guessed at — an intentional scope boundary, not an oversight.
5. **What this chunker does *not* catch, on purpose:** the CommonJS pattern `module.exports = class Application {}` (found in `koa`, an older-style library) is a plain assignment expression, not one of the four recognized declaration shapes — it's invisible to this chunker today. That's a documented, real limitation, not a silent bug: `koa`'s main class currently contributes zero chunks.

#### REAL BUG THIS SHAPE CAUGHT

The originally-planned `tree-sitter-languages` package called an old tree-sitter API that no longer exists in the installed version. Swapped to the actively-maintained, per-language `tree-sitter-javascript`/`tree-sitter-typescript` packages instead, and verified real parsing worked before writing a single line of chunking logic on top.



```
src/codesee/chunking/docs_chunker.py
```

## docs\_chunker.py — prose has no functions, so it splits differently

READMEs and markdown docs don't have a function/class structure to preserve — instead, this splits on sentence boundaries and packs sentences into token-capped, overlapping chunks. Also home to a real fix found through live evaluation: stripping badge/HTML noise before chunking.

```

_HTML_TAG = re.compile(r"<[>]+>")
_BADGE_LINE = re.compile(r"^s*(\![^\[]*\)\([^\)]*\)\!|([^\[]*\)\([^\)]*\)\s*$",

def _strip_badge_and_html_noise(text: str) → str:
    """READMEs conventionally open with a block of CI/coverage/PyPI badges and
    raw HTML alignment wrappers. That block embeds as a short, generic,
    surprisingly high-scoring chunk that beats genuinely relevant code for
    almost any query -- found by diagnosing why a docs chunk kept outranking
    the actual implementation in eval results."""
    text = _BADGE_LINE.sub("", text)
    text = _HTML_TAG.sub(" ", text)
    return re.sub(r"\n{3,}", "\n\n", text)

def split_into_sentences(text: str) → list[str]:
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    sentences: list[str] = []
    for paragraph in paragraphs:
        sentences.extend(s for s in _SENTENCE_BOUNDARY.split(paragraph) if s)
    return sentences

def chunk_doc_text(text: str, repo: str, path: str) → list[DocChunk]:
    text = _strip_badge_and_html_noise(text)
    sentences = split_into_sentences(text)
    if not sentences:
        return []

    packed = pack_units(sentences, MAX_CHUNK_TOKENS, OVERLAP_RATIO, joiner=" ")
    return [
        DocChunk(repo=repo, path=path, chunk_index=i, text=chunk_text)
        for i, (_, _, chunk_text) in enumerate(packed)
    ]

```

## Walkthrough

1. **`_BADGE_LINE`** **regex** matches a whole line that's *only* a markdown badge image, either bare (`![Coverage](url)`) or link-wrapped (`[[](link-URL)]`)

`url))` — the two shapes shields.io-style CI/coverage/PyPI badges take.

`re.MULTILINE` makes `^/$` match line boundaries, not just start/end of the whole string.

2. `_HTML_TAG` strips anything in angle brackets — `<p align="center">`, ``, `</p>` — replacing each with a single space so words on either side don't get glued together.
3. **Order matters: badges first, then HTML.** A badge line is often itself wrapped in `<p align="center"> ... </p>` — stripping the badge pattern first, then the surrounding HTML tags, cleanly removes both layers.
4. **Splitting is paragraph-aware, not just sentence-aware.** `split_into_sentences` first breaks on blank lines (`\n\n` = a paragraph boundary) and only then splits each paragraph into sentences via `_SENTENCE_BOUNDARY` — a regex lookbehind for `./!/?` followed by whitespace.
5. **Same `pack_units` engine as the code chunkers**, but with `OVERLAP_RATIO = 0.15` (code chunks default to no overlap unless oversized) and `joiner=" "` instead of `"\n"` — sentences read naturally joined by spaces, unlike code lines.

#### THE DIAGNOSIS BEHIND THIS FIX

Live eval on a freshly-indexed repo (`typer`) kept returning a README's very first chunk as the top result for unrelated implementation questions. Inspecting that chunk directly showed it was almost entirely CI badge markup, not prose — confirmed the fix mattered by checking the chunk's *content* before and after, not just re-running the eval and hoping.

## Part 4 — Embeddings

Turning chunk text into the numbers a vector database can compare.

## registry.py — which two models, and why

```
GENERAL = "general"
CODE = "code"

def build_default_embedders() → dict[str, SentenceTransformerEmbedder]:
    return {
        GENERAL: SentenceTransformerEmbedder("sentence-transformers/all-MiniLM-L6-v2", dimensi
        # jinaai/jina-embeddings-v2-base-code was tried first but its custom remote
        # code imports a transformers.pytorch_utils symbol removed in transformers v5 --
        # this model is a standard architecture with no custom code, so it isn't
        # exposed to that breakage.
        CODE: SentenceTransformerEmbedder(
            "flax-sentence-embeddings/st-codesearch-distilroberta-base", dimensions=768
        ),
    }
```

### Walkthrough

1. **Two named constants (GENERAL, CODE)** instead of raw strings scattered everywhere — every other module (the API, the eval harness, the Streamlit UI) imports these same two names, so renaming a model key later means touching one file, not grep-and-replace across the codebase.
2. **Different dimensions per model (384 vs. 768) is completely normal** — each model has its own fixed output size, and because they're stored in *separate* Qdrant collections (`codeseek__general` vs `codeseek__code`), they never need to be compared to each other directly.
3. **The comment documents a real dead end.** The original plan (and the project report) called for `jina-embeddings-v2-base-code`. It's broken against the installed `transformers` version — found and fixed *before* writing any chunker code on top of it, exactly the kind of dependency-drift issue worth checking early.

## service.py — the two embedding entry points

```
class EmbeddingService:
    def __init__(self, embedders: dict[str, Embedder], query_cache_size: int = 256):
        self._embedders = embedders
        self._query_cache: OrderedDict[tuple[str, str], list[float]] = OrderedDict()
        self._query_cache_size = query_cache_size
        self.cache_hits = 0
        self.cache_misses = 0

    def embed_all(self, texts: list[str]) → dict[str, list[list[float]]]:
        return {key: embedder.embed(texts) for key, embedder in self._embedders.items()}

    def embed_one(self, model_key: str, text: str) → list[float]:
        key = (model_key, text)
        cached = self._query_cache.get(key)
        if cached is not None:
            self.cache_hits += 1
            self._query_cache.move_to_end(key)
            return cached

        self.cache_misses += 1
        vector = self._embedders[model_key].embed([text])[0]
        self._query_cache[key] = vector
        if len(self._query_cache) > self._query_cache_size:
            self._query_cache.popitem(last=False)
        return vector
```

### Walkthrough

1. **Two methods, two very different jobs.** `embed_all` is the *indexing*-side path — batch-embed a list of chunk texts through *every* configured model, used once per chunk ever (each chunk's text is unique, so there's nothing to cache). `embed_one` is the *query*-side path — embed a single question through *one specific* model, used every time someone searches.
2. `embed_one` is an LRU cache, hand-rolled with `OrderedDict`. The cache key is `(model_key, text)` — the same question against the general model and the code model are different cache entries, correctly. On a hit, `move_to_end` marks that entry as most-recently-used. On a miss past the size cap, `popitem(last=False)` evicts the *oldest* entry (the one at the front of the dict) — that combination is what makes it "least-recently-used" eviction, not just FIFO.
3. **Measured, not assumed: this cache gives a real 15× speedup** on a repeated identical question (21.2ms → 1.4ms) — a search UI or an eval run re-asking the same

thing benefits directly.

4. `cache_hits`/`cache_misses` as plain instance attributes — simple counters, not a metrics framework, but enough to let `scripts/cost_measurement.py` report a real before/after number instead of a guess.

## Part 5 — Storage & Retrieval

Where chunks live, and how a question finds them.

## schema.py — what a stored chunk looks like, and its ID

```
_ID_NAMESPACE = uuid.UUID("5b6a8f0c-6b3f-4a1a-9b8b-6b7c9b9e6f10")

@dataclass(frozen=True)
class ChunkPayload:
    repo: str
    language: str
    path: str
    symbol_name: str
    symbol_type: str
    start_line: int
    end_line: int
    text: str
    license: str = ""
    stars: int = 0

    def point_id(self) → str:
        key = f"{self.repo}:{self.path}:{self.symbol_name}:{self.start_line}:{self.end_line}"
        return str(uuid.uuid5(_ID_NAMESPACE, key))

    def as_payload_dict(self) → dict:
        return {"repo": self.repo, "language": self.language, "path": self.path,
                "symbol_name": self.symbol_name, "symbol_type": self.symbol_type,
                "start_line": self.start_line, "end_line": self.end_line,
                "text": self.text, "license": self.license, "stars": self.stars}

def collection_name(corpus_name: str, model_key: str) → str:
    return f"{corpus_name}__{model_key}"
```

### Walkthrough — the most important five lines in the whole codebase

1. **`point_id()` is deterministic, not random.** `uuid.uuid5` generates a UUID by hashing a namespace plus a string — the *same* input string always produces the *same* UUID. The input string here is `repo:path:symbol_name:start_line:end_line` — the chunk's own identity.
2. **Why this matters: idempotent re-indexing, for free.** If you index `httpx` today and index it again tomorrow with unchanged code, every chunk computes the exact same ID both times — Qdrant's `upsert` overwrites the existing point instead of creating a duplicate. No "check if this already exists" logic needed anywhere else in the codebase; it falls out of this one function.
3. **The flip side, honestly documented in `docs/scaling_design.md`:** if a function's line numbers shift (code added above it) but its body is otherwise unchanged, it gets

a *new* ID, and the old point becomes an orphan. The scaling design's answer is to delete-then-reinsert a whole changed file's chunks rather than trying to diff at the chunk level — simpler to reason about than tracking every possible chunk move.

4. **`_ID_NAMESPACE`** is a **fixed, arbitrary UUID** — its only job is to make sure this project's UUIDs don't collide with some other system's UUID5s built from similar strings. It never changes.
5. **`collection_name("codeseek", "general")`** → **`"codeseek__general"`**. One tiny function, but it's the single place that string-formatting convention lives — every other module calls this instead of hand-building the string, so the naming scheme can't drift.

## qdrant\_store.py — the only file that talks to Qdrant directly

A thin wrapper class. Every other module that needs to touch the vector database goes through `QdrantStore`'s methods — nothing else imports `qdrant_client` directly.

```
def build_metadata_filter(repo: str | None, language: str | None) → models.Filter | None:
    conditions = []
    if repo:
        conditions.append(models.FieldCondition(key="repo", match=models.MatchValue(value=repo))
    if language:
        conditions.append(models.FieldCondition(key="language", match=models.MatchValue(value=language))
    return models.Filter(must=conditions) if conditions else None

class QdrantStore:
    def __init__(self, path=None, url=None, location=None, timeout=None):
        if url:
            self._client = QdrantClient(url=url, timeout=timeout)
        elif path:
            self._client = QdrantClient(path=path)
        else:
            self._client = QdrantClient(location=location or ":memory:")

    def vector_search(self, collection, query_vector, limit, query_filter=None) → list[SearchHit]:
        if not self._client.collection_exists(collection):
            return []
        results = self._client.query_points(
            collection_name=collection, query=query_vector, limit=limit, query_filter=query_filter
        )
        return [SearchHit(id=str(p.id), score=p.score, payload=p.payload or {}) for p in results]

    def keyword_search(self, collection, query_text, limit, extra_filter=None) → list[SearchHit]:
        """Pure payload filter match, no vector involved -- Qdrant scores filter-only
        matches as 1.0, which the retrieval layer replaces with the vector floor score."""
        if not self._client.collection_exists(collection):
            return []
        text_filter = models.Filter(should=[
            models.FieldCondition(key="text", match=models.MatchText(text=query_text)),
            models.FieldCondition(key="symbol_name", match=models.MatchText(text=query_text)),
        ])
        must = [text_filter, extra_filter] if extra_filter else [text_filter]
        points, _ = self._client.scroll(collection_name=collection, scroll_filter=models.Filter(must=must))
        return [SearchHit(id=str(p.id), score=0.0, payload=p.payload or {}) for p in points]
```

### Walkthrough



1. **Three constructor modes, one class.** `path=` (on-disk, single-process, used for day-to-day dev), `url=` (a real Qdrant server, used for load testing and by anything needing concurrent access), `location=":memory:"` (ephemeral, used by every test in the suite so tests never touch real disk state).
2. **`collection_exists` guards, everywhere.** Every search method checks this first and returns an empty list rather than letting Qdrant raise on a not-yet-created collection — so searching a corpus that hasn't been indexed yet returns "no results," not a 500 error.
3. **`vector_search` is genuine meaning-based search** — it hands the query's embedding straight to Qdrant's `query_points`, which runs the actual similarity search (HNSW-accelerated, in server mode).
4. **`keyword_search` is a completely different kind of query** — no vector involved at all, just a payload filter (`MatchText`) checking whether the raw query string appears in the chunk's `text` or `symbol_name`, via `scroll` (a filter-only fetch, not a similarity ranking). Every hit gets `score=0.0` here on purpose — the real score gets assigned later, in `merge_hits`.
5. **`build_metadata_filter` is a free function, not a method** — both `/search` and `HybridRetriever` build the same kind of repo/language filter from it, so the filtering logic exists in exactly one place.

## hybrid.py — merge\_hits, the whole hybrid-search idea in ~15 lines

```
def merge_hits(vector_hits: list[SearchHit], keyword_hits: list[SearchHit], top_k: int) → list[SearchHit]:
    by_id: dict[str, MergedHit] = {}
    for hit in vector_hits:
        hit.id: MergedHit(id=hit.id, score=hit.score, payload=hit.payload, source="vector")

    floor_score = min((hit.score for hit in vector_hits), default=0.0)

    for hit in keyword_hits:
        existing = by_id.get(hit.id)
        if existing is None:
            by_id[hit.id] = MergedHit(id=hit.id, score=floor_score, payload=hit.payload, source="keyword")
        else:
            by_id[hit.id] = replace(existing, source="both")

    merged = sorted(by_id.values(), key=lambda h: h.score, reverse=True)
    return merged[:top_k]
```

### Walkthrough — read this one line by line, it's short and it matters

1. **Start with every vector hit, keyed by ID.** `by_id` is a dict, so looking up "did this chunk already show up?" is O(1) instead of scanning a list.
2. **`floor_score = min(vector scores, default=0.0)`.** This single line is the entire "additive, never overriding" policy: the weakest score any real vector match earned becomes the score a keyword-only hit gets assigned. Not zero (which would sink it to the bottom), not the top score (which would let a coincidental keyword match outrank a genuinely strong semantic result) — the floor.
3. **The loop over keyword hits does one of two things per hit:** if this chunk wasn't already found by vector search, add it fresh with the floor score and `source="keyword"`. If it *was* already found, keep its real (usually higher) vector score untouched and just relabel `source="both"` — `dataclasses.replace` makes a new frozen instance with one field changed, since `MergedHit` is immutable.
4. **Sort once, at the end, then truncate to `top_k`.** Simple, and correct because every score in `by_id` is now on the same scale (Qdrant cosine scores, or the borrowed floor score) — nothing needs a second normalization pass.

5. **This is why a literal symbol name like `JWTBearer` can be found** even when its embedding alone wouldn't win: keyword search finds the exact string match, and the merge step lets it surface without needing to out-compete on cosine similarity at all.

#### UNDER ACTIVE INVESTIGATION

This exact function is the natural place to add a docs-vs-code score damping factor — the RAG-tuning fix being proposed alongside this book. A one-line multiplier on `score` for `symbol_type == "doc"` chunks, applied right where scores are already being assigned, before the final sort.

## Part 6 — Orchestration

The one function that ties chunking, embedding, and storage into a single indexing pass.

## pipeline/index.py — index\_repo, the whole pipeline in one function

```
def index_repo(repo_path, repo, store, embedding_service, corpus_name, batch_size=64) → int:
    chunks: list[ChunkPayload] = []
    skipped_files = 0

    with timed_stage("extract_and_chunk", repo=repo.name):
        for record in walk_repo(repo_path, repo.name, repo.language):
            try:
                chunks.extend(_file_to_chunks(repo_path, record, repo))
            except (OSError, UnicodeError) as exc:
                skipped_files += 1
                logger.warning("Skipping unreadable file %s/%s: %r", repo.name, record.path, exc)

    total = 0
    for start in range(0, len(chunks), batch_size):
        batch = chunks[start : start + batch_size]

        with timed_stage("embed", repo=repo.name, batch_size=len(batch)):
            vectors_by_model = embedding_service.embed_all([c.text for c in batch])

        with timed_stage("upsert", repo=repo.name, batch_size=len(batch)):
            for model_key, vectors in vectors_by_model.items():
                collection = collection_name(corpus_name, model_key)
                store.ensure_collection(collection, vector_size=len(vectors[0]))
                store.upsert_chunks(collection, batch, vectors)

        total += len(batch)

    return total
```

### Walkthrough

1. **Two distinct phases, both wrapped in `timed_stage`**. Phase 1 (extract + chunk) walks every file and turns it into `ChunkPayload` objects, entirely in memory, no network calls. Phase 2 (embed + upsert) is batched — `batch_size=64` chunks at a time — and is where the real time goes, since embedding is CPU-bound work.
2. **Per-file error isolation**. The `try/except` is *inside* the per-file loop, not around the whole phase — one corrupt or unreadable file logs a warning and the loop continues; it doesn't abort the other 999 files in the repo. This was tested directly in `test_failure_modes.py` by monkeypatching `Path.read_text` to fail for one specific file.

3. **Why batching, not one chunk at a time.** Both embedding models process a batch of texts in one call — far more efficient than calling the model once per chunk (measured directly: batch size 32 gave 14.3 chunks/sec vs. 12.0 at batch size 1).
4. `store.ensure_collection(..., vector_size=len(vectors[0]))` — the vector dimension isn't hardcoded anywhere; it's read off the first embedding actually produced. If a model were swapped for one with a different output size, this line wouldn't need to change.
5. **Two collections get written per batch** (one per model), inside the same `timed_stage("upsert", ...)` block — both models' data for a batch land before moving to the next batch, so the two collections never drift out of sync with each other mid-run.

#### CALLED FROM

`scripts/build_index.py` (loops this over every repo in `CORPUS`),  
`scripts/index_one.py` (one ad-hoc repo via CLI), and `api/app.py`'s `/ingest` route (one ad-hoc repo via HTTP, no separate process needed).

## Part 7 — API

The one file every UI and CLI script ultimately talks to.

## api/app.py — create\_app, and why it's a factory

```
def create_app(store: QdrantStore, embedding_service: EmbeddingService, corpus_name=CORPUS_NAME):
    app = FastAPI(title="CodeSeek")

    @app.get("/repos", response_model=ReposResponse)
    def list_repos() → ReposResponse:
        collection = collection_name(corpus_name, "general")
        repos = sorted(store.list_repos(collection), key=lambda r: r[1], reverse=True)
        return ReposResponse(repos=[RepoInfo(name=n, chunk_count=c) for n, c in repos])

    @app.post("/search", response_model=SearchResponse)
    def search(req: SearchRequest) → SearchResponse:
        results_by_model, timings_by_model = {}, {}
        query_filter = build_metadata_filter(req.repo, req.language)

        for model_key in req.models:
            collection = collection_name(corpus_name, model_key)

            t0 = time.perf_counter()
            query_vector = embedding_service.embed_one(model_key, req.query)
            t1 = time.perf_counter()
            vector_hits = store.vector_search(collection, query_vector, limit=req.top_k, query_filter=query_filter)
            t2 = time.perf_counter()
            keyword_hits = store.keyword_search(collection, req.query, limit=req.top_k, extra_filter=query_filter)
            t3 = time.perf_counter()
            hits = merge_hits(vector_hits, keyword_hits, req.top_k)
            t4 = time.perf_counter()
            # ... build StageTimings and SearchResultItem lists from hits ...

            results_by_model[model_key] = hits
            timings_by_model[model_key] = (t1 - t0, t2 - t1, t3 - t2, t4 - t3)

        return SearchResponse(results_by_model=results_by_model, timings_by_model=timings_by_model)

    return app
```

### Walkthrough

1. `create_app` is a function that **returns an app**, rather than one global `app = FastAPI()` built at import time. It takes `store` and `embedding_service` as parameters. This is the single design choice that makes the whole API testable in milliseconds: every test in `test_api.py` calls `create_app(fake_store, fake_embedder)` with an in-memory store and a deterministic fake embedder — no real models loaded, no real disk touched, 50 tests run in under 10 seconds.
2. `/search` deliberately does **not** call `HybridRetriever.search` as one opaque function. It inlines the same three steps (`vector_search` → `keyword_search` → `merge_hits`) by hand, with a `time.perf_counter()` reading between each —

that's the only way to produce the per-stage-latency numbers the UI's timing panel shows. `HybridRetriever` still exists and is used by the eval harness, which doesn't need per-stage timing.

3. **The loop runs once per requested model** (`for model_key in req.models`) — a single `/search` call can return both the general and code models' rankings for the exact same query, which is what the UI's side-by-side comparison tabs are built from.
4. **`/repos` sorts by chunk count, descending** — the UI's "live index" badge list shows the biggest repos first, a minor but deliberate UX choice.
5. **Every route is a plain `def`, not `async def`**. FastAPI runs synchronous route functions in a background thread pool automatically — since nothing here does async I/O (the embedding models and Qdrant client are all synchronous), this is simpler than fighting with async wrappers for no benefit, and it's why a long-running `/ingest` call doesn't block `/health` from answering concurrently.

## Part 8 — Evaluation

How "is the search actually good" gets turned into a number.

## eval/harness.py — run\_eval, and the trickiest correctness detail in the project

```
def _is_relevant(payload: dict, item: GroundTruthItem) → bool:
    # An oversized function/class is split into "Name#part1", "Name#part2", ...
    # by both chunkers -- any part counts as finding the target symbol.
    symbol_name = payload.get("symbol_name", "")
    name_matches = symbol_name == item.symbol_name or symbol_name.startswith(f"{item.symbol_name}#")
    return payload.get("repo") == item.repo and payload.get("path") == item.path and name_matches

def run_eval(ground_truth, store, embedding_service, corpus_name, model_keys, top_k=10) → list[EvalResult]:
    retriever = HybridRetriever(store)
    results: list[EvalResult] = []

    for model_key in model_keys:
        collection = collection_name(corpus_name, model_key)
        recall5, recall10, rr = [], [], []

        for item in ground_truth:
            query_vector = embedding_service.embed_one(model_key, item.question)
            hits = retriever.search(collection, query_vector, item.question, top_k=top_k)
            ranked_ids = [h.id for h in hits]
            relevant_ids = {h.id for h in hits if _is_relevant(h.payload, item)}

            recall5.append(recall_at_k(ranked_ids, relevant_ids, 5))
            recall10.append(recall_at_k(ranked_ids, relevant_ids, 10))
            rr.append(reciprocal_rank(ranked_ids, relevant_ids))

        n = len(ground_truth)
        results.append(EvalResult(model_key=model_key, recall_at_5=sum(recall5)/n,
                                  recall_at_10=sum(recall10)/n, mrr=sum(rr)/n, num_questions=n))

    return results
```

### Walkthrough

1. `_is_relevant` was a real, silent bug for a while. The first version compared `symbol_name == item.symbol_name` exactly. But an oversized class like `DigestAuth` gets split into `DigestAuth#part1`, `#part2`, etc. by the chunker — the plain name `"DigestAuth"` never appears in the store at all once a symbol is split. Every ground-truth question targeting a large class scored a permanent, silent zero regardless of retrieval quality, until this `.startswith(f"{item.symbol_name}#")` check was added.



2. **No repo filter, on purpose, even though we know the answer's repo.** A real user asking "how does X work" doesn't tell the system which of the 24 repos to look in — filtering the eval query to the known-correct repo would measure something easier than what a real user experiences, and quietly inflate the numbers.
3. **Recall and MRR are computed from the exact same `hits` list** — one retrieval call per question per model, not one call per metric. `relevant_ids` is derived *from* the hits (which of the returned results match the ground truth), not looked up independently — if the correct chunk isn't in the top-`k` at all, it's simply absent from `relevant_ids`, and every metric correctly scores that as a miss.
4. **The averaging happens once, after the full loop** — `sum(recall5) / n` — so every question contributes equally regardless of how many chunks that repo has.

#### IN YOUR OWN WORDS

"Recall@k asks 'was the right answer anywhere in my top k results' — a yes/no averaged over every question. MRR asks 'how close to #1 was it' —  $1 \div \text{rank}$ , so #1 scores 1.0, #2 scores 0.5, and 'not found at all' scores 0. Recall tells you if the system can find it; MRR tells you if it's actually useful once found."

## Part 9 — Observability & UI

## observability/timing.py — timed\_stage

```
@contextmanager
def timed_stage(stage: str, **fields):
    start = time.perf_counter()
    error = None
    try:
        yield
    except Exception as exc:
        error = repr(exc)
        raise
    finally:
        duration_ms = round((time.perf_counter() - start) * 1000, 2)
        record = {"stage": stage, "duration_ms": duration_ms, **fields}
        if error is not None:
            record["error"] = error
            logger.error(json.dumps(record))
        else:
            logger.info(json.dumps(record))
```

### Walkthrough

1. **@contextmanager** turns a generator function into a **with**-block. Everything before **yield** runs on entry; everything in **finally** runs on exit, success or failure, which is exactly "start a timer, do the work, always log how long it took."
2. The **try/except/finally** shape logs a failure and still re-raises it. **error = repr(exc)** captures what went wrong for the JSON log line, then **raise** (bare, no argument) re-raises the exact same exception — the caller still sees the real error; this block only adds a structured log entry alongside it, never swallows anything.
3. **\*\*fields** lets every call site attach its own context — **timed\_stage("embed", repo=repo.name, batch\_size=len(batch))** — without **timed\_stage** itself needing to know what fields any particular stage cares about.
4. The log message is a JSON string, and the handler format is just **"%(message)s"**. That's what makes every log line grep-able and parseable as one JSON object per line — no log-parsing library needed to answer "how long did every embed stage take across this run."

## streamlit\_app/app.py — the UI is a thin HTTP client, nothing more

Worth understanding one thing above all: the Streamlit app contains *zero* retrieval logic. It only ever calls `GET /repos`, `POST /search`, and `POST /ingest` over HTTP and renders the JSON it gets back.

```
@st.cache_data(ttl=5)
def fetch_repos(url: str) → list[dict] | None:
    try:
        resp = httpx.get(f"{url}/repos", timeout=10.0)
        resp.raise_for_status()
        return resp.json()["repos"]
    except httpx.HTTPError:
        return None

# ... later, on search submit ...
payload = {
    "query": query, "models": ["general", "code"], "top_k": int(top_k),
    "repo": repo_filter if repo_filter in repo_names else None,
    "language": language_filter or None,
}
resp = httpx.post(f"{api_url}/search", json=payload, timeout=30.0)
```

### Walkthrough

1. `@st.cache_data(ttl=5)` means `fetch_repos` only actually hits the network once every 5 seconds, no matter how many times Streamlit re-runs the script (which it does on almost every interaction) — keeps the "live index" list feeling instant without hammering the API.
2. **Returning `None` on any `HTTPError`**, then checking `if repos is None:` `st.error(...); st.stop()` right after the call, is the entire "backend isn't running" error path — one clear message instead of a raw stack trace.
3. **The search payload's `repo` field** is `repo_filter` only if it's actually one of the real fetched `repo_names` — the dropdown's placeholder option ("All indexed repos") deliberately isn't a real repo name, so this comparison naturally sends `None` (no filter) when nothing specific is selected, with no separate special-case check.
4. **The rest of the file (not shown here) is presentation only** — CSS for the dark GitHub-inspired theme, hand-drawn inline SVG icons instead of emoji, and the two-

column layout. None of it changes what gets searched or how; swap the whole UI for a different frontend tomorrow and the API underneath doesn't need to change.

#### WHY THIS SEPARATION MATTERS

Because the UI is "just" an HTTP client, everything it can do, a curl command or a different frontend could do identically. That's not an accident — it's what a factory-built, fully-tested API layer underneath is *for*.

Every code block above is the real, current source — copied directly from the files as they exist in the project right now, not simplified or paraphrased. Read the actual files alongside this if you want the parts trimmed for length (full docstrings, imports, type hints).